AI • MACHINE LEARNING • DJANGO

ML PIPELINE DEPLOYED

Student Placement Predictor (PlaceAI)

Predictive Career Placement Engine with Scikit-Learn Model Integration

PROJECT NAME	APP NAME	ML INTEGRATION	MODELS LOADED
Student_placement	**Studentapp**	**Joblib / Scikit-Learn**	**StandardScaler & Classifier**

Executive Summary: The Student Placement Prediction system combines Django's robust web framework with Machine Learning predictive analytics. Candidates register and log in to submit academic metrics (Cumulative CGPA, Cognitive IQ index, and Extracurricular Profile Score). The server pipeline normalizes inputs via a pre-trained `scalermodel.pkl` and classifies placement likelihood using `mlmodel.pkl` to provide immediate diagnostic feedback.

1. Data Architecture (`Studentapp/models.py`)

The persistence tier tracks registered student candidates with unique primary keys, contact information, and credential storage.

Model	Field	Type / Definition	Functional Purpose
Student	`S_Id`	CharField(max_length=50, primary_key=True)	Unique student enrollment identifier
	`S_Name`	CharField(max_length=100)	Student full legal name
	`S_Pass`	CharField(max_length=100)	Authentication password
	`S_Email`	EmailField(max_length=100)	Candidate official communication email
	`S_No`	CharField(max_length=20, null=True, blank=True)	Emergency or mobile telephone number

```python
from django.db import models

# Create your models here.
class Student(models.Model):
    S_Id = models.CharField(max_length=50, primary_key=True)
    S_Name = models.CharField(max_length=100)
    S_Pass = models.CharField(max_length=100)
    S_Email = models.EmailField(max_length=100)
    S_No = models.CharField(max_length=20, blank=True, null=True)

    def __str__(self):
        return self.S_Name
```

2. Application Routing (`Student_placement/urls.py`)

The centralized URL configuration connects auth flows, registration handlers, and the predictive ML endpoints.

Route URL	Handler Function	Route Name	Purpose & Operation
`''`	`views.home`	`'home'`	Landing portal with feature overview
`'register/'`	`views.register`	`'register'`	Student onboarding form
`'save_student/'`	`views.save_student`	`'save_student'`	POST action for candidate persistence
`'login/'`	`views.login`	`'login'`	Candidate sign-in portal
`'login_validation/'`	`views.login_validation`	`'login_validation'`	POST auth verification & session set
`'predict/'`	`views.predict`	`'predict'`	Diagnostic metric entry interface
`'result/'`	`views.result`	`'result'`	ML inference pipeline execution & report
`'logout/'`	`views.logout`	`'logout'`	Session termination

```python
from django.contrib import admin
from django.urls import path
from Studentapp import views

urlpatterns = [
    path('', views.home, name='home'),
    path('register/', views.register, name='register'),
    path('save_student/', views.save_student, name='save_student'),
    path('login/', views.login, name='login'),
    path('login_validation/', views.login_validation, name='login_validation'),
    path('predict/', views.predict, name='predict'),
    path('result/', views.result, name='result'),
    path('logout/', views.logout, name='logout'),
    path('admin/', admin.site.urls),
]
```

3. Controller & ML Inference Pipeline (`Studentapp/views.py`)

The view module encapsulates user authentication, multi-field query parsing, and scikit-learn model loading via `joblib`.

> **Machine Learning Inference Breakdown:**
>
> - `BASE_DIR` resolution binds `scalermodel.pkl` and `mlmodel.pkl` reliably.
> - Feature vector `[cgpa, iq, profile_score]` is ingested from GET query parameters.
> - Inputs are preprocessed with `scaler.transform([list_vals])` to guarantee standard feature distribution.
> - Classifier yields binary prediction `ans = int(mlmodel.predict(scaled_input)[0])` (1 = Placed, 0 = Not Placed).

```python
import os
from django.shortcuts import render
from django.http import HttpResponse, HttpResponseRedirect
from django.urls import reverse
from django.db.models import Q
from Studentapp.models import Student
import joblib

# Resolve model paths
BASE_DIR = os.path.dirname(os.path.dirname(os.path.abspath(__file__)))
SCALER_PATH = os.path.join(BASE_DIR, 'scalermodel.pkl')
MODEL_PATH = os.path.join(BASE_DIR, 'mlmodel.pkl')

def home(request):
    return render(request, 'home.html')

def register(request):
    return render(request, 'register.html')

def save_student(request):
    if request.method == 'POST':
        s_id = request.POST.get('S_Id') or request.POST.get('Student_Id')
        s_name = request.POST.get('S_Name') or request.POST.get('Student_Name')
        s_pass = request.POST.get('S_Pass') or request.POST.get('password')
        s_email = request.POST.get('S_Email') or request.POST.get('Student_Email')
        s_no = request.POST.get('S_No') or request.POST.get('Student_No', '')

        if not s_id or not s_name:
            return render(request, 'register.html', {'error': 'Student ID and Name are required!'})

        already_exists = Student.objects.filter(S_Id=s_id).exists()
        if already_exists:
            return render(request, 'register.html', {'error': 'Student with this ID already exists!'})

        student = Student(
            S_Id=s_id,
            S_Name=s_name,
            S_Pass=s_pass,
            S_Email=s_email,
            S_No=s_no
        )
        student.save()
        return HttpResponseRedirect(reverse('login'))
    return HttpResponseRedirect(reverse('register'))

def login(request):
    return render(request, 'login.html')

def login_validation(request):
    if request.method == 'POST':
        s_name = request.POST.get('S_Name') or request.POST.get('username')
        s_pass = request.POST.get('S_Pass') or request.POST.get('password')

        student = Student.objects.filter(S_Pass=s_pass).filter(Q(S_Name=s_name) | Q(S_Id=s_name)).first()
        if student:
```

```
            request.session['student_name'] = student.S_Name
            request.session['student_id'] = student.S_Id
            return HttpResponseRedirect(reverse('predict'))
        else:
            return render(request, 'login.html', {'error': 'Invalid Username/ID or Password!'})
    return HttpResponseRedirect(reverse('login'))

def logout(request):
    request.session.flush()
    return HttpResponseRedirect(reverse('home'))

def predict(request):
    student_name = request.session.get('student_name', '')
    return render(request, 'predict.html', {'student_name': student_name})

def result(request):
    scaler = joblib.load(SCALER_PATH)
    mlmodel = joblib.load(MODEL_PATH)

    cgpa = float(request.GET.get('cgpa', 0))
    iq = float(request.GET.get('iq', 0))
    profile_score = float(request.GET.get('profile_score', 0))

    list_vals = [cgpa, iq, profile_score]
    scaled_input = scaler.transform([list_vals])
    ans = int(mlmodel.predict(scaled_input)[0])

    student_name = request.session.get('student_name', '')

    context = {
        'ans': ans,
        'list': list_vals,
        'cgpa': cgpa,
        'iq': iq,
        'profile_score': profile_score,
        'student_name': student_name
    }
    return render(request, 'result.html', context)
```

4. Live Application Output Screenshots

Below are full-resolution visual artifacts showing the end-to-end user workflow: onboarding, candidate login, evaluation form, and ML prediction output.

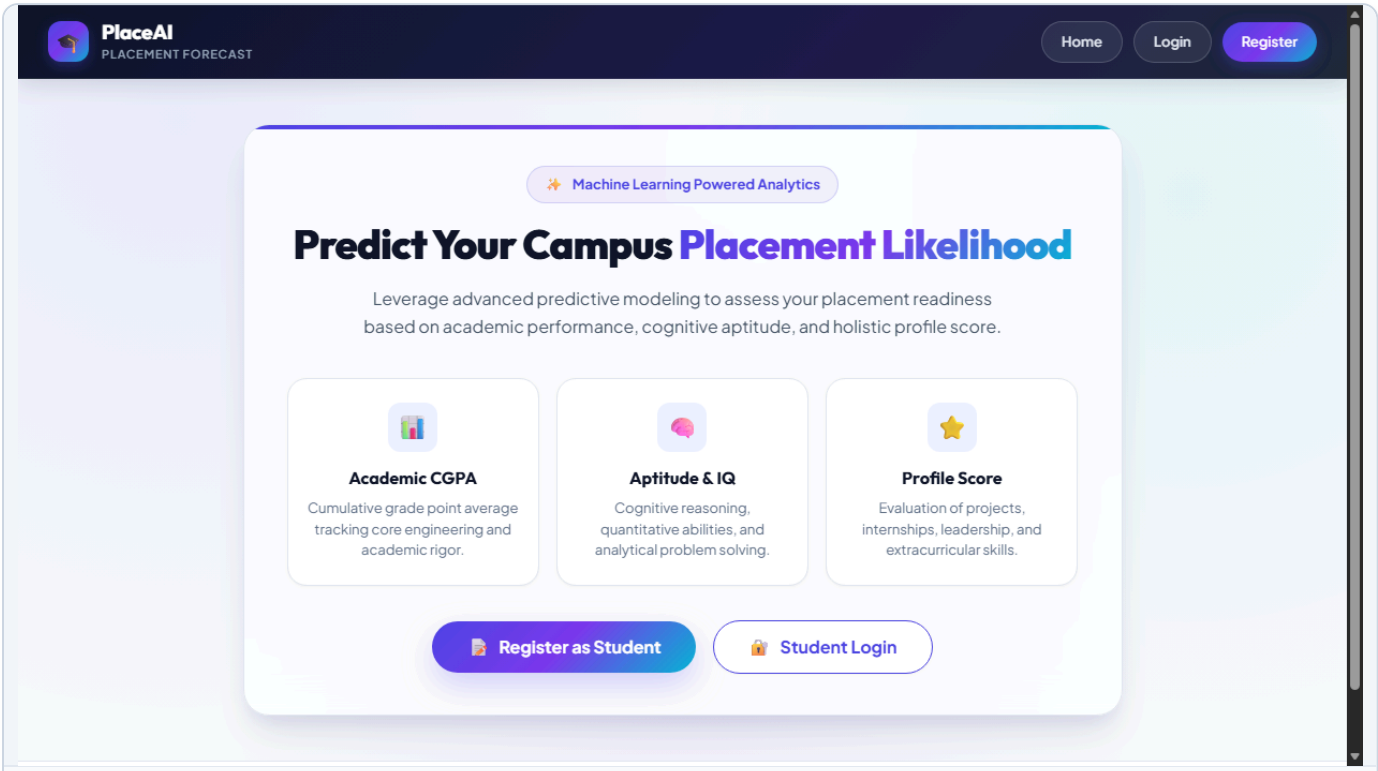


Figure 3.1: PlaceAI Homepage with portal navigation, call-to-actions, and placement analysis introduction.

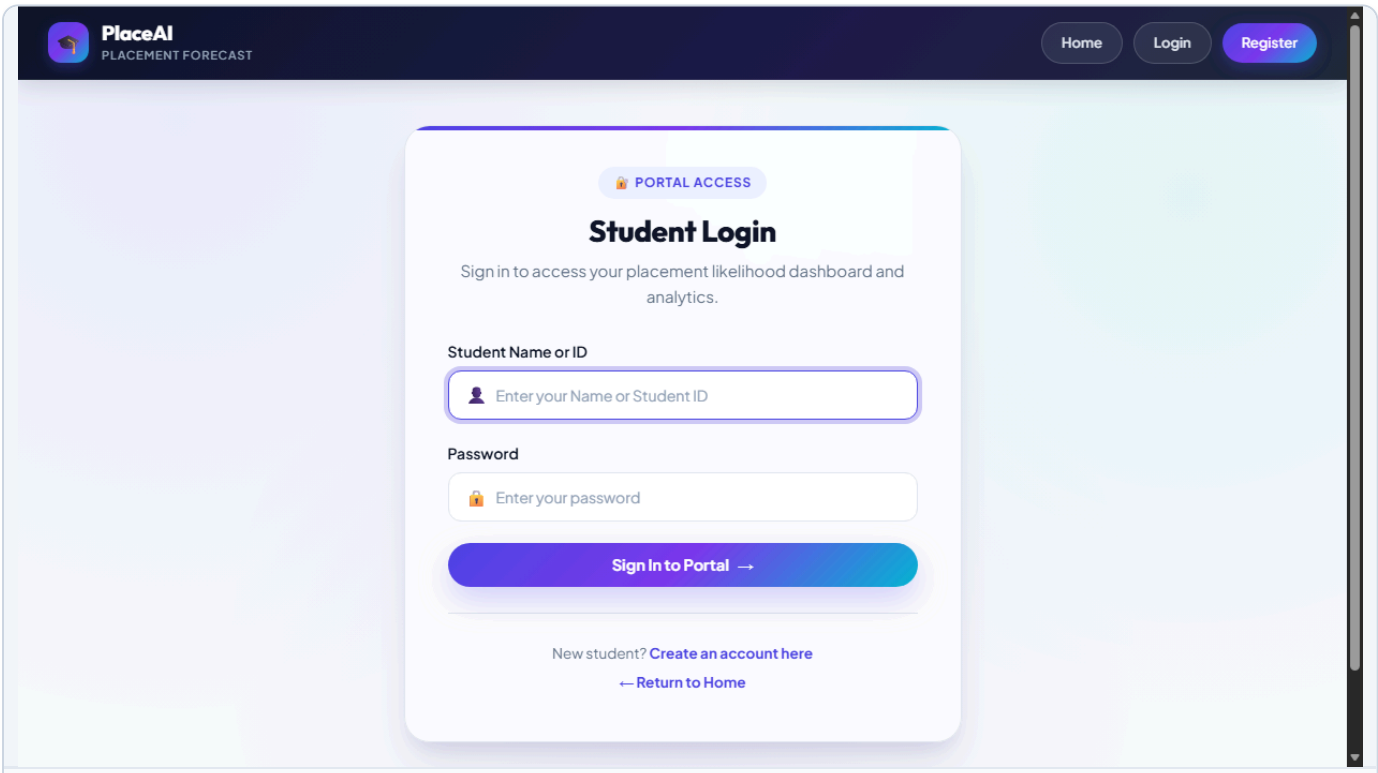


Figure 3.2: Student Authentication Gateway accepting Username / Student ID and security credentials.

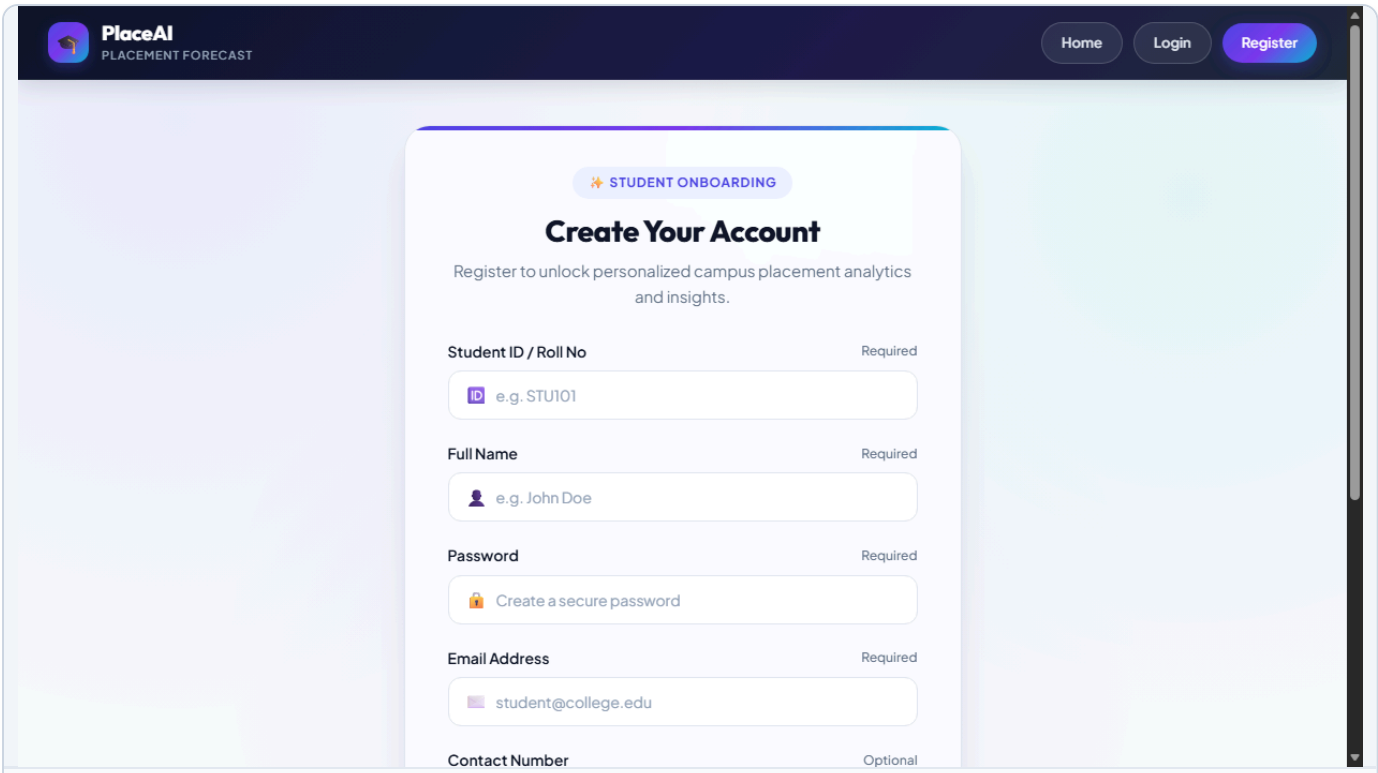


Figure 3.3: Student Registration Portal validating unique ID, Name, Email, and Phone records.

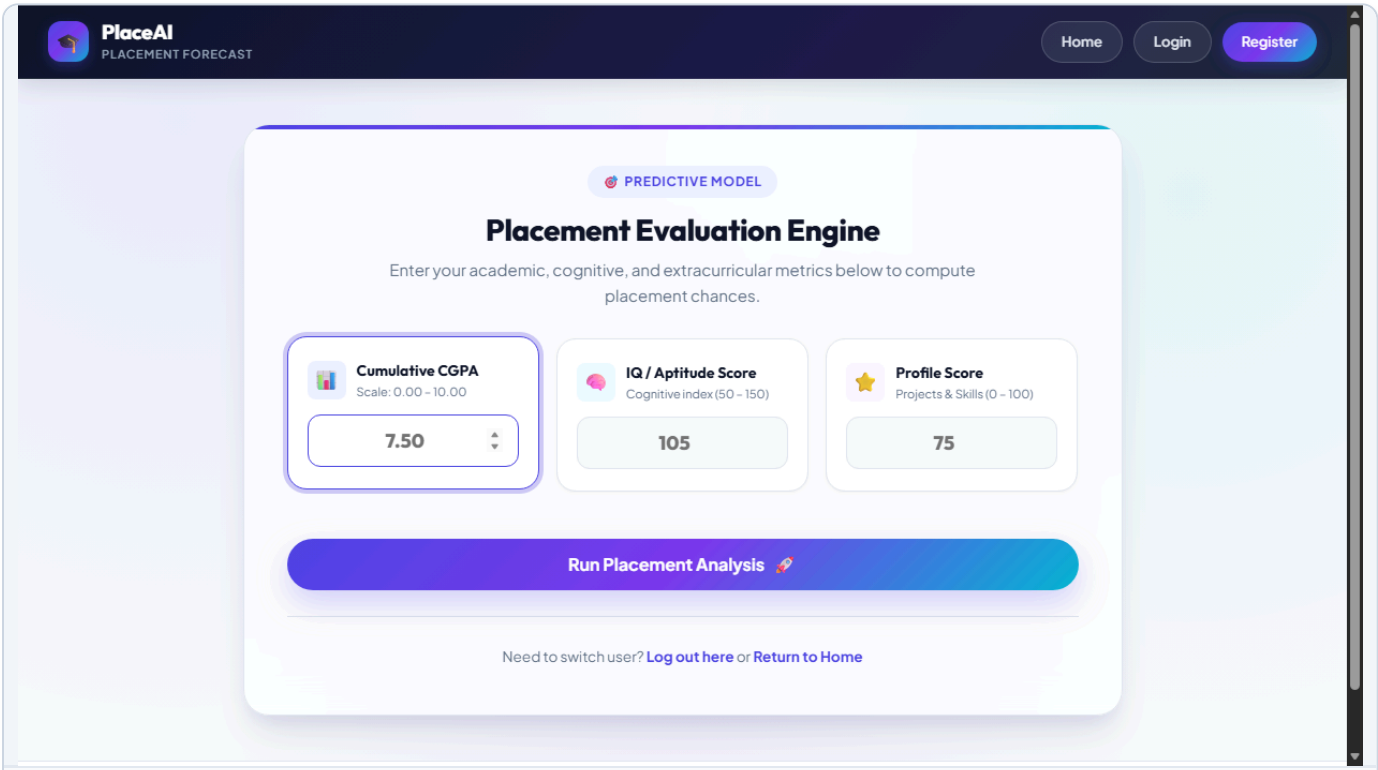


Figure 3.4: Evaluation Engine Form allowing students to input CGPA, Cognitive IQ score, and Extracurricular Profile Score.

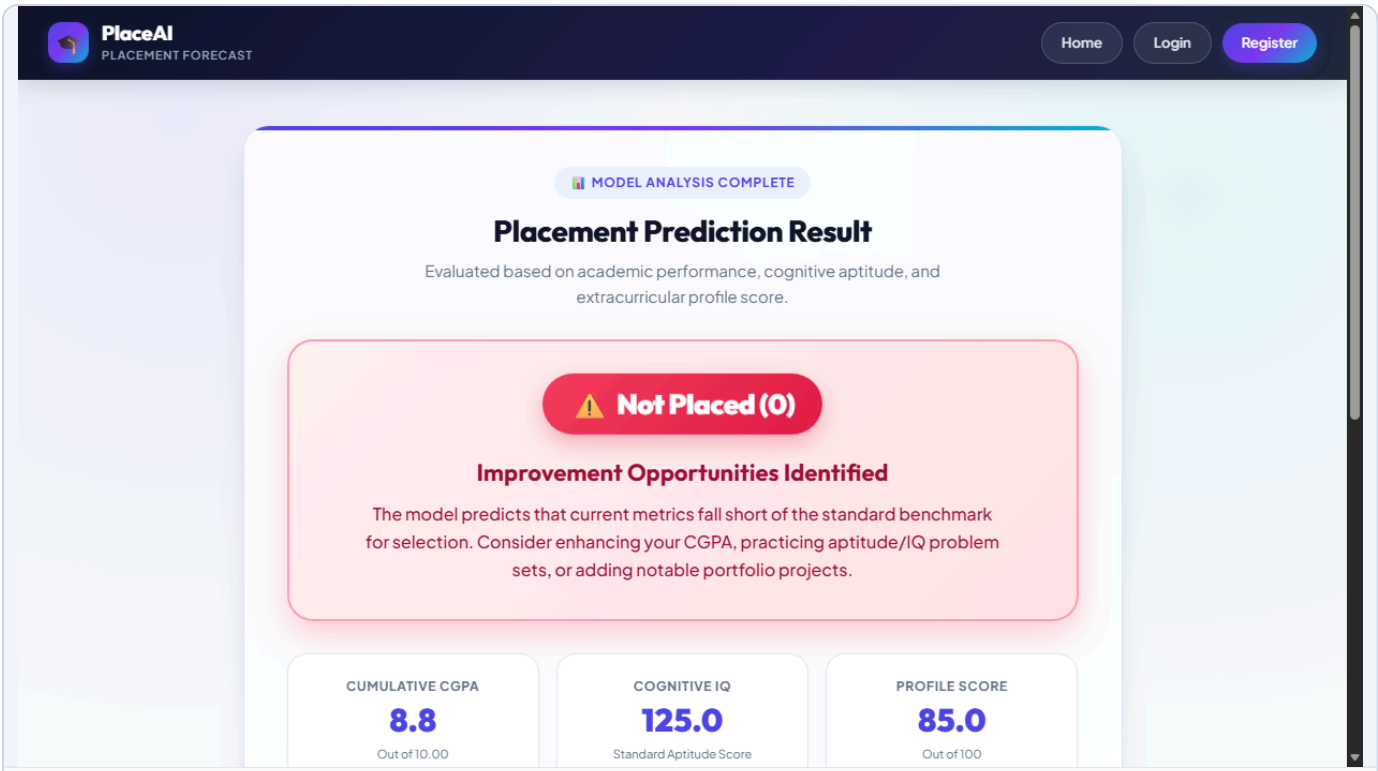


Figure 3.5: Real-Time ML Classification Output displaying the computed placement decision, candidate feedback, and metric recap.